# Analyse des achats Black Friday avec Azure

Rapport de démarche et observations

**Réalisé par : Ayman TALHA**

Dans ce projet, j'ai utilisé Python, Azure et Power BI pour explorer un jeu de données d'achats. Mon objectif était de partir d'un fichier CSV, de contrôler les données, de les stocker dans Azure et de présenter les résultats dans un rapport interactif. J'ai aussi testé trois services d'IA sur des exemples de produit, d'avis et d'image.

## 1 Choix des données et préparation avec Python

J'ai utilisé le fichier train.csv du jeu Black Friday de sdolezel sur Kaggle. Il contient des informations sur les clients, les produits et les montants des achats. J'ai travaillé sur cette seule source. Les avis et l'image utilisés pour les essais d'IA sont des exemples séparés, ils ne viennent pas de ce CSV.

### Découverte du fichier

J'ai chargé le CSV dans Jupyter avec pandas. J'ai affiché les premières lignes, les types des colonnes, les statistiques et les valeurs manquantes. Le fichier de départ contient 550 068 lignes et 12 colonnes. La colonne Purchase correspond au montant associé à chaque ligne d'achat.

Avant de nettoyer, j'ai recherché les lignes entièrement identiques avec duplicated().sum(). Le résultat était de zéro. J'ai aussi vérifié Purchase : aucun montant manquant et aucun montant inférieur ou égal à zéro. En revanche, j'ai trouvé 173 638 valeurs absentes dans Product_Category_2 et 383 247 dans Product_Category_3.

### Mes choix de nettoyage

J'ai conservé les catégories secondaires manquantes, car je n'avais pas d'information pour les compléter. Mettre zéro aurait ajouté un code de catégorie qui n'était pas présent dans la source. Dans Python, j'ai utilisé le type Int64, qui accepte les entiers et les valeurs manquantes. Dans SQL, ces colonnes acceptent NULL.

J'ai ajouté Gender_Label et Marital_Status_Label pour obtenir des libellés plus lisibles. J'ai remplacé 4+ par « 4 ans ou plus » pour la durée de résidence. J'ai conservé les tranches d'âge déjà présentes dans le fichier.

J'ai également ajouté Transaction_ID, un numéro technique unique par ligne. Il sert de clé primaire dans SQL. Ce numéro ne représente pas un panier réel et ne permet pas, à lui seul, de repérer des achats identiques.

J'ai exporté black_friday_clean.csv dans data/processed, sans ajouter l'index de pandas. Le fichier nettoyé contient toujours 550 068 lignes, avec 15 colonnes. J'ai placé la vérification de ses colonnes après l'export pour pouvoir reprendre le notebook dans l'ordre.

# 2 Stockage dans Azure et organisation du SQL

J'ai regroupé les ressources dans rg-blackfriday. J'ai utilisé le compte de stockage stblackfriday241095, avec une redondance LRS et le niveau Hot, puis déposé le CSV nettoyé dans le conteneur privé blackfriday-data. Blob conserve le fichier ; Azure SQL conserve ensuite ses données sous forme de table.

J'ai retenu Azure SQL plutôt que Cosmos DB parce que mes données sont tabulaires. La base db-blackfriday est hébergée sur le serveur sql-blackfriday-241095. J'ai numéroté les scripts selon leur ordre d'utilisation : 1 pour le schéma, 2 pour l'import depuis Blob, 3 pour les contrôles et 4 pour les vues. Pour une base vide, je crée la table, j'importe le CSV, je vérifie les données. Les vues constituent un complément séparé.

### 1 Le schéma avec 01_schema.sql

J'ai créé dbo.BlackFriday avec la commande CREATE TABLE conservée dans 01_schema.sql. Les 15 colonnes suivent l'ordre du CSV nettoyé. J'ai choisi des nombres entiers pour les montants et les codes numériques, du texte pour les identifiants produit et les tranches d'âge, et du texte Unicode pour les libellés. Les catégories 2 et 3 acceptent NULL. Ce bloc correspond à la requête SQL query 1 exportée depuis Azure ; il sert à créer la table quand elle est absente.

### 2 L'import depuis Blob avec 02_import_blob.sql

Pour lire le conteneur privé, j'ai utilisé un jeton SAS. J'ai préparé la clé principale, puis l'autorisation BlackFridaySAS et la source externe BlackFridayBlob. Le fichier 02_import_blob.sql rassemble ces blocs dans leur ordre de dépendance, puis la commande BULK INSERT. Ils correspondent aux onglets SQL query 4, SQL query 3 et SQL query 5. Le numéro des onglets Azure ne représente pas leur ordre d'exécution.

FIRSTROW = 2 ignore les noms des colonnes. CODEPAGE = 65001 correspond à UTF-8 et KEEPNULLS conserve les valeurs manquantes. Après BULK INSERT, j'ai compté les lignes de la table. Les secrets sont remplacés par des paramètres dans le fichier du rendu. Ces blocs sont à utiliser dans une nouvelle base préparée pour l'import, sans les relancer sur la table déjà remplie.

### 3 Les contrôles avec 03_quality_checks.sql

Le fichier 03_quality_checks.sql reprend les vérifications exportées depuis Azure : SELECT COUNT(*) pour compter les lignes et SELECT TOP (5), avec un tri par Transaction_ID, pour afficher un aperçu. Elles correspondent aux onglets SQL query 2 et SQL query 6. Le comptage de la table a donné 550 068 lignes, comme le CSV. Les contrôles détaillés des montants, des valeurs manquantes et des doublons sont présentés dans le notebook Python.

### 4 Les vues dans 04_views.sql

Une vue enregistre une requête que l'on peut réutiliser. Le fichier propose une vue détaillée, une vue des indicateurs, une analyse par âge et une analyse par produit. SUM additionne les montants, AVG calcule la moyenne et GROUP BY rassemble les lignes d'un même groupe. La somme utilise BIGINT pour éviter un dépassement de capacité ; la moyenne utilise un type décimal.

Ces vues sont un complément conservé dans le dossier de scripts. Les six requêtes exportées depuis Azure ne contiennent pas de création de vue : je ne les présente donc pas comme exécutées lors de l'import. Mon rapport Power BI utilise directement dbo.BlackFriday.

### La vérification depuis Jupyter

J'ai connecté Python à SQL avec pyodbc et ODBC Driver 18. Après un blocage de la connexion interactive, j'ai utilisé DeviceCodeCredential pour me connecter à Microsoft avec un code. La requête de comptage a confirmé les 550 068 lignes. J'ai ensuite fermé la connexion.

# 3 Visualisation et observations dans Power BI

J'ai utilisé la table BlackFriday dans Power BI. Le modèle repose sur une seule table, donc aucune relation entre tables n'était nécessaire. J'ai construit les mesures de comptage, de clients distincts, de somme et de moyenne, puis vérifié les résultats avec les calculs du notebook.

| Indicateur | Résultat |
|---|---|
| Lignes d'achat | 550 068 |
| Clients distincts | 5 891 |
| Produits distincts | 3 631 |
| Montant total des achats | 5 095 812 742 |
| Montant moyen par ligne d'achat | 9 263,97 |
| Montant médian par ligne d'achat | 8 047 |

Je n'ai pas ajouté de devise sans confirmation de la source. J'ai remplacé « CA » par « Montant total des achats » et « Panier moyen » par « Montant moyen d'un achat ». Le fichier ne contient pas d'identifiant permettant de reconstituer un panier complet.

## La page Vue d'ensemble

J'ai présenté les indicateurs dans quatre cartes, puis ajouté les montants par âge, la répartition par genre et les montants par catégorie principale. Le filtre d'âge permet de voir comment les résultats changent selon le groupe sélectionné. J'ai testé ce filtre et ajusté la présentation pour faciliter la lecture.

J'ai remarqué que les 26–35 ans représentent 219 587 lignes et un montant total de 2 031 770 578, soit environ 39,9 % du montant global. Ce groupe contribue le plus au total, mais son montant moyen, environ 9 252,69, n'est pas le plus élevé. Cela m'a montré qu'un total important peut surtout venir d'un grand nombre de lignes d'achat.

Sur la répartition des montants par genre, les hommes représentent environ 76,72 % et les femmes 23,28 %. Ces pourcentages portent sur les montants enregistrés, pas sur la proportion de clients distincts. Ils ne suffisent pas non plus à expliquer les causes de cette différence.

La catégorie principale 1 arrive en tête des montants avec 1 910 013 754, devant les catégories 5 et 8. Pourtant, la catégorie 5 comporte plus de lignes : 150 933 contre 140 378 pour la catégorie 1. J'ai donc comparé les nombres de lignes et les montants moyens au lieu de regarder uniquement le classement par total.

## La page Analyse détaillée

J'ai créé une matrice produit × tranche d'âge avec un dégradé de couleur, pour répondre à la demande de heatmap. Je l'ai limitée aux dix produits ayant le montant total le plus élevé afin de garder un tableau lisible. Le produit P00025442 est premier avec 27 995 166 sur 1 615 lignes d'achat.

Dans le notebook, j'ai aussi observé que la catégorie de ville B a le total le plus élevé, tandis que C a le montant moyen le plus élevé. J'ai gardé les codes A, B et C sans leur attribuer des noms de villes que je ne connais pas.

# 4 Essais des services Azure IA

J'ai utilisé ces essais pour comprendre ce que les services peuvent apporter à un projet de commerce. Les exemples sont indépendants des achats du CSV. Je n'ai pas relié les résultats de l'IA aux produits Kaggle, car je ne dispose pas de leurs descriptions, de leurs avis ou de leurs photos.

## Azure OpenAI

J'ai utilisé la ressource aoai-blackfriday-241095 et le modèle gpt-4.1-mini. J'ai fourni les caractéristiques d'un casque fictif : Bluetooth, couleur noire, réduction de bruit et autonomie de 30 heures. J'ai demandé une description courte en français, sans inventer de caractéristique ni de promotion.

Le service a produit un texte marketing, mais j'ai remarqué des formulations sur le confort et le design qui n'étaient pas directement justifiées par les informations fournies. Même avec une consigne précise, il faut donc relire le résultat avant de le publier.

J'ai également demandé le résumé d'un avis fictif. Il reprend les points positifs sur la batterie et la réduction de bruit, ainsi que les problèmes de confort et de livraison. Ce type de résumé pourrait aider à lire un ensemble d'avis, à condition de vérifier qu'il conserve les remarques importantes.

## Azure Language

Avec lang-blackfriday-241095, j'ai testé trois avis fictifs en français. Le premier, satisfait du son, a été classé positif. Le deuxième, parlant d'un produit défectueux et d'une livraison en retard, a été classé négatif. Le troisième, décrivant simplement un casque noir avec Bluetooth, a été classé neutre.

J'ai ensuite extrait les expressions clés. Le service a notamment relevé « casque », « son », « livraison » et « retard ». J'ai vu que ces deux fonctions se complètent : le sentiment indique une tonalité, tandis que les expressions clés aident à repérer les sujets évoqués. Les scores expriment la confiance du modèle, pas une certitude sur tous les avis possibles.

## Azure Computer Vision

Avec vision-blackfriday-241095, j'ai envoyé une photographie de casque. Le résultat contient la description « a pair of headphones » et des étiquettes comme headphones et electronics. Les scores affichés sont d'environ 98,4 % pour headphones et 99,5 % pour electronics.

J'ai utilisé une image JPEG après avoir rencontré un problème de format. J'ai aussi essayé le français : la description demandée par cette fonction ne le prend pas en charge, alors que les étiquettes peuvent être disponibles en français. J'ai gardé l'anglais pour l'appel qui demande les deux résultats.

Cet essai reconnaît le contenu visible de l'image. Il ne prouve pas la marque, l'autonomie ou la qualité du casque. Pour refaire l'analyse du notebook, l'image produit.jpg doit être conservée avec celui-ci. Vision Studio permet également de tester une image directement dans une interface web Azure.

## 5 Difficultés rencontrées et ce que je retiens

J'ai rencontré une erreur parce que la variable clean n'avait pas encore été créée. Cela m'a aidé à comprendre que les cellules dépendent les unes des autres et doivent être exécutées dans l'ordre.

Lors de l'export, j'ai rencontré une PermissionError. Un CSV ouvert dans Excel peut empêcher son écriture ; il faut le fermer et vérifier les droits du dossier. J'ai aussi corrigé une cellule qui lisait le CSV nettoyé avant sa création.

Pour Azure OpenAI, j'ai obtenu une erreur 401 liée à l'authentification. J'ai vérifié que la clé et le point de terminaison correspondaient à la même ressource. Pour SQL, j'ai rencontré une erreur 40613 signalant une indisponibilité de la base, sans pouvoir confirmer sa cause exacte. La connexion Microsoft dans le navigateur s'est aussi bloquée ; la méthode par code a permis de poursuivre.

J'ai retiré du notebook les essais de connexion en échec et les vérifications ODBC répétées pour garder une démarche plus claire. J'ai conservé une explication des problèmes. J'ai également distingué le jeton SAS de l'URL SAS et corrigé les noms des indicateurs dans Power BI.

En changeant de PC, j'ai constaté que les textes des onglets SQL n'étaient pas forcément retrouvés avec le compte Azure. La base et ses données sont dans Azure, mais les scripts doivent aussi être conservés en fichiers .sql. Cela explique l'intérêt du dossier de scripts dans le rendu.

## 6 Applications possibles et limites

Dans un contexte professionnel, je pourrais utiliser cette analyse pour comparer les segments de clientèle et repérer les produits qui contribuent le plus aux montants des achats. Avant de proposer une action, je regarderais aussi le nombre de lignes et le montant moyen. Un total élevé ne signifie pas automatiquement qu'un produit est plus rentable : je n'ai pas les coûts ni les marges.

Pour aller plus loin, je croiserais les achats avec des avis réels et un catalogue produit. Language pourrait aider à repérer les thèmes récurrents des retours clients. OpenAI pourrait proposer des descriptions à relire, et Vision aider à classer des images. Il faudrait disposer de liens fiables entre ces nouvelles données et les identifiants produit.

Pour prévoir les ventes ou travailler sur les stocks, il me faudrait des dates, des quantités, des niveaux de stock et plusieurs périodes comparables. Le fichier actuel ne permet pas de construire une prévision dans le temps. Une publication dans Power BI Service pourrait faciliter le partage si l'accès au service est disponible ; mon travail vérifié porte sur le rapport Desktop.

Je retiens surtout que le contrôle des données doit accompagner chaque étape. J'ai appris à passer d'un CSV à une table SQL, puis à des visualisations, et à expliquer pourquoi certains choix sont nécessaires. Les services d'IA apportent des exemples utiles, mais leurs résultats demandent aussi une vérification.

### Sources et fichiers du projet

Consignes : Projet_data_M2_BDD-2.pdf.

Données : Black Friday, sdolezel, Kaggle (kaggle.com/datasets/sdolezel/black-friday).

Calculs et sorties : notebook du projet. Scripts : 01_schema.sql, 02_import_blob.sql, 03_quality_checks.sql et 04_views.sql.

Restitution : fichier Power BI et rapport dans le dépôt GitHub du projet.